

AI-Driven Software Development

Summer Semester 2026



Final Project: Time Tracking Application (Part I)

Publication: 2026-06-08

Requirements update: 2026-06-29

Final submission (project & report): 2026-07-19 23:59 CEST

Prof. Dr. G. Fraser,
Benedikt Fein

1 Introduction

For your final project, you will develop a time tracking web application. The main target audience of the application are not businesses but individuals (e.g. students, freelancers, researchers) who want to track the time they spent working on their lectures, seminars, projects, or other activities. Similar CLI-based applications are *timewarrior*¹ or *watson*².

Similar to the ToDo app project from the weekly assignments, this will be a full-stack web application. The frontend must be implemented using a JavaScript or TypeScript framework of your choice. The backend must be developed in a JVM-based language and include both an API layer and a connection to a data persistence layer. You can choose any database implementation as long as it can run locally (e.g. H2, SQLite, PostgreSQL, MySQL, MongoDB).

Before starting your implementation, carefully consider the overall architecture of your application. While mockups are not mandatory, it is highly recommended to engage in a common structured approach to software engineering, such as design thinking or domain-driven design and create prototypes to plan your user interface and user experience effectively.

2 Application Requirements

The application must fulfil the requirements below. In three weeks (2026-06-29), we will publish some updated and additional requirements that will be part of the same application (i.e. part II of the final project) to simulate having changing requirements during a real-world development process.

In the following, *tasks* refer to individual tracked time slots. *Projects* can be lectures, seminars, or other activities (cf. Introduction). As part of your implementation, you can choose to keep the theme general, or adapt the vocabulary used within your application to a specific user group. In case you want to extend the requirements to adapt the application to your chosen target audience, you may do so, as long as the core requirements outlined below are met.

¹<https://timewarrior.net/>

²<https://jazzband.github.io/Watson/>

2.1 Functional Requirements

- **User Management:** Users must be able to register, log in, log out, and change their password.
- **Time Tracking:** Users should be able to easily track the time spent on projects. The main action to track time is to start or stop recording a task at the current moment. It should also be possible to add or delete tasks later-on. Tracked tasks can also be changed after they have been recorded, e.g. to move the start time in case the user forgot to start the activity at the correct time.
- **Projects:** The user can mark individual tasks as being related to one or more projects. The set of available projects can be defined by the user. When changing a task, it should also be possible to change which project(s) it is associated with. Projects should have a hierarchy where each project can optionally include one or more subprojects.

The system tracks the total time spent on a project by summing up the time spent on tasks on a project and its subprojects. For example, assuming this lecture is a project in the system, one hour spent working on the final assignment of this lecture also counts as one hour spent on the lecture as a whole. In case a project P has subprojects a and b , and a task is associated with both subprojects a and b , then it should however still only count once towards the parent project P .

- **Task Overview:** When looking at a project, it should be possible for the user to see all tracked tasks associated with the project and its subprojects. The application should give an overview of how much total time was spent on a project within a user-selectable time frame. As a more general overview, the application should be able to show the tracked tasks of the current day, week, and month. The overviews should be presented suitably in tabular or calendar-like form.
- **Data Persistence:** All event data must be stored in a database and remain accessible after leaving the page and logging back in later. Time tracking for a task should continue if the browser or application is stopped.

2.2 Non-Functional Requirements

- **Usability:** The user interface should be clear, intuitive, and modern. Frequently performed actions in the application should be achievable with few user interactions to make the application as convenient as possible.
- **Security:** The application must validate user input, hash passwords securely, and use safe authentication methods (e.g. sessions or tokens).
- **Performance:** The application should load within one second on a device with a stable internet connection.
- **Independence:** The application must be deployable fully locally (including database and all supporting services) without access to other cloud-based resources. In case the

application depends on other supporting services, these must be deployable as part of the same unit (e.g. Docker Compose) as the main application.

3 Organisational Requirements and Grading

The project is an *individual* assignment. You are *not* allowed to work in groups. You are explicitly allowed and even encouraged to use suitable AI-/LLM-based tools for all development steps. The tool choice is *not* limited to tools we presented in the lecture. Instead, you are free to choose any tools you deem suitable.

3.1 Development Process

While designing and implementing the features of the application, you *must* follow an iterative development process. Concretely:

- You *must* track the requirements as issues. These issues should, if suitable, follow the user story format with proper description, tasks, and acceptance criteria.
- You should also open new issues for other project-related tasks, e.g. for found bugs or planned refactorings.
- Create separate pull requests or commits for self-contained changes and at least one per issue. I.e. create pull requests or separate commits not only for new features, but also for example for refactorings, bug fixes, or additional tests.
- Use a common-sense approach of when to use issues/pull-requests vs. when to make a quick fix-up commit directly. For example, in case of single-line typo or test fixes, the overhead from issues/pull-requests does not make sense, and you can commit directly to main. The goal is that you can in the end demonstrate that you followed an iterative development process where you actively thought about the individual requirements, their implementation, and the verification.
- Link pull requests to relevant issues if applicable.³ This helps you to reconstruct the development process when writing the report and us when grading.
- Since this is an individual project, you will of course not find other developers that can review your code before merging the pull request. You may use the automatic AI-based review tools instead.

Your application should be tested by automatic test suites including unit, integration, and system tests. You should have a line coverage of at least 90 % for both the backend and frontend parts of the system. Keep in mind that successful tests that cover the acceptance criteria of a user story are usually part of the ‘definition of done’.

³<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue>

To ensure the coding guidelines you defined are upheld, you should define a continuous integration pipeline. This pipeline should run the test suites, linters, formatting checkers, and other automatic tools you want to use. You should aim for a ‘green’ pipeline on the default branch and only merge pull requests with passing pipelines.

3.2 Written Project Report

In addition to the application itself, you *must* submit a written report. The report must be added to the repository in the `report/` directory. The main goal of this report is that you critically reflect on your specific experiences with the AI-driven software development process. The recommended structure of the report is:

1. System architecture and design decisions
Give a brief overview of the high-level design decisions you made while designing and implementing the application requirements. For example, you can explain technical decisions (e.g. system architecture, frontend framework choice), or in case there were requirements for which the specification allowed for multiple different approaches, explain why you chose the one you implemented. You can also explain the decisions that went into the overall design of the user interface.
2. Used LLMs and AI-based tools
Summarise which LLMs and AI-based tools you used and how you used them. Explain for which development scenarios you used which tools.
E.g. prompting model *a* via the web-chat for requirements elicitation, agent tool *b* with models *c* and *d* for testing, ...
3. Where did the AI-tools work well?
You should describe concrete examples, scenarios, and/or development phases where the LLM/AI-tool you used performed well.
4. Where did the AI-tools not work as intended?
Similar as above for the case where your chosen tools worked well, also describe cases where the tool did not work as well as expected. Please describe how you adapted your usage of the LLMs to get more useful responses. Give concrete examples of issues you faced and how you adapted, e.g. the prompt, to resolve it.
5. Missing AI-tools (optional)
Are there scenarios where you would have wanted to use another (AI-based) tool that can (partially) automate stuff for you, but the tool does not exist yet? If so, please describe the scenario and how the envisioned tool could help you. You can also mention tools that exist already, but you were not able to use, e.g. due to a complicated setup and/or the tool’s pricing model.

There are no fixed word count or formatting requirements for this report. A succinct discussion of the relevant information in your own words is very much preferred over padded-out

AI-generated text.⁴ You can submit the report for example as Markdown, AsciiDoc, or LaTeX⁵ files directly in the repository in the `report/` directory. For all explanations and examples, please link relevant issues, pull requests, or commits, if applicable, to help us better understand the report in the context of your implementation and development process.

3.3 Submission

The whole project *must* be developed and submitted via the repository provided via GitHub classroom:

<https://classroom.github.com/a/TuXr0YKT>

The submission deadline is Sunday, 2026-07-19 23:59 CEST. You will no longer have write access to the repository afterwards. Ensure you pushed all of your code and the report by the deadline since this is a hard deadline. We will *not* accept late submissions via other channels. All artefacts (i.e. source code, documentation, issues) including the report *must* be part of the repository itself. Only the last state of the default branch of your repository (i.e. the default state GitHub shows when looking at a repository) will be considered for grading. Issues and pull requests (open and closed) in the GitHub repository count as being part of the repository.

You *must* add a file `WHOAMI.txt` at the top-most level of the repository in the following format:

GitHub username: ACCOUNT-NAME

Name: Your real full name (as shown e.g. on Stud.IP)

University Email: ???@ads.uni-passau.de

Matriculation number: 000000

Without this file we cannot match your username to your real name and the matriculation number and cannot enter the grade in the campus portal. Submissions without this file will not be graded.

Please create a `.gitignore` file⁶ in your repository that prevents common build, test, and IDE artefacts (e.g. `node_modules/`⁷, `build/`, `target/`, `dist/`, coverage reports) from being committed and pushed.

3.4 Grading Environment

To be able to grade your submission, we have to check whether the required functionality of the application works as expected. For that, we need to be able to run the application and its test suite on our machine. We will not be able to grade non-executable submissions. Provide a `README.md` file in your repository that explains the required tools, SDKs, and setup steps to be able to do run the application and its test suite(s).

Your application and its supporting infrastructure (e.g. database) *must* be executable using a container environment (e.g. Docker, Docker Compose, Podman). As an alternative to containers,

⁴Since the AI cannot critically reflect for you, it will also not achieve the intended goal in the latter case.

⁵For LaTeX please also add the compiled PDF either in the repo or as CI artefact.

⁶<https://git-scm.com/docs/gitignore>

⁷Especially `node_modules`, I do not want to have to Git-clone multiple 100s of megabytes of JS libraries 45 times.

you may package it using Nix⁸ and ensure a reproducible development, build, and execution environment that way.

The test suite execution does not need to be containerised. You can assume the following tools are installed on our systems:

- Java JDK 21 and 25
- NodeJS LTS version 24 or newer
- Standard build tools/package managers: Maven, Gradle, NPM, Yarn (and other build tools like Vite, Webpack, ... that are automatically pulled in as dependencies of the project using the primary build tool/package manager)
- Firefox browser
- Docker/Podman with Docker Compose to build and run containers
- Nix with Flakes enabled
- Standard Unix command line tools (e.g. bash, grep, sed, ...)

If we need tools not mentioned here to build and run your application, you *must* ask us *before* the submission deadline about that, preferably via a *non-anonymous* question in the StudIP forum or in one of the Q&A sessions. Only then can we figure out if using the tool is okay, or if that would block us from positively grading your submission.

3.5 Grading

To decide on the final grade for this assignment and by extension the whole course, we will take into account the following points, including but not limited to: successful submission of the weekly assignments for the ToDo-app, fulfilment of the requirements outlined in Section 2, a development process following the requirements from Section 3.1, your implementation (e.g. following a sensible architecture that matches the one described in your report), sufficient tests for the application (cf. Section 3.1), and content and quality of the written report (cf. Section 3.2).

To give you some orientation, for last year's lecture we used the following grading criteria for the development process and the report. For the weekly assignments and the fulfilment of the requirements, the grading steps were roughly speaking 'missing', 'partially existing/working', and 'fully working'.

3.5.1 Development Process

Issues Issues were not only created for the requested high-level features, but also for, e.g. bug-fixes or additional smaller features that came up during development.

Pull Requests/Commits Separate PRs/Commits for features, refactorings, and bug fixes referencing relevant issues were created. PRs/Commits usually implement one thing.

⁸<https://nix.dev/>

CI Pipeline Runs linters, unit tests for both server and client, and system tests.

Unit Tests Achieves the required coverage thresholds for both the server and client components.

System Tests Both the happy path and relevant edge- and failure-cases are tested for important system features.

3.5.2 Report

System Architecture & Design Solid explanation of system architecture, design decisions, technology stack and why they were chosen. Alternatives and reasons why they were not chosen are discussed.

Used LLMs and AI-based tools Each tool's use is specifically described, including how it was prompted/integrated/configured, what it contributed, and why it was appropriate for that context.

Where did AI tools work well? Provides clear, specific examples of scenarios or development phases where AI tools excelled. Describes why the tools worked well in each case.

Where did AI tools not work? Provides specific, well-explained examples where AI tools failed or underperformed. Describes how you recognised the issue and how you adapted your approach.